



# INF01195 | Actividad 3

## Optimización paralela en GPU del problema extendido de la mochila mediante CUDA

**Fecha de entrega:** domingo 14 de junio de 2026

**Porcentaje de la actividad:** 20 % de la asignatura

**Lenguaje sugerido:** CUDA C++ (se permite C++ para la versión secuencial y CUDA C++ para las versiones paralelas).

**Modalidad de evaluación:** informe técnico y presentación con defensa, ejecución del programa y explicación técnica mediante PPTX.

### 1. Objetivo de la actividad

Implementar, paralelizar y analizar un algoritmo genético para resolver una versión extendida del problema de la mochila utilizando CUDA. El foco principal de la actividad no es solamente obtener una solución de alto valor, sino demostrar comprensión sobre qué partes del algoritmo genético pueden ejecutarse eficientemente en GPU, qué limitaciones aparecen por la arquitectura CUDA y cómo evaluar correctamente el impacto de la paralelización en términos de tiempo, *speed-up*, uso de memoria, escalabilidad y calidad de solución.

La actividad considera dos componentes evaluados:

- **Informe técnico 30 %:** formulación, metodología experimental, resultados, análisis de rendimiento, análisis de memoria y conclusiones.
- **Presentación, defensa, ejecución y explicación técnica del código 70 %:** explicación oral del diseño, resultados, decisiones técnicas, ejecución del programa y explicación de las partes principales de la implementación mediante la presentación.

El código no tendrá una nota separada fuera de la presentación. Será evaluado durante la defensa mediante una demostración funcional, los resultados generados por el programa, la explicación técnica incluida en la PPTX y preguntas sobre decisiones de programación, memoria, paralelismo, aleatoriedad, restricciones y medición. No se exigirá abrir el proyecto completo ni revisar archivos extensos de código durante la exposición; si el grupo desea mostrar código, deberá hacerlo mediante fragmentos breves insertos en la presentación.

### 2. Contexto del problema

El problema clásico de la mochila consiste en seleccionar un subconjunto de ítems, donde cada ítem posee un valor y un peso, de manera que el valor total sea máximo sin exceder la capacidad de la mochila.

Sea un conjunto finito  $N$  de  $n$  ítems. Cada ítem  $i$  tiene un valor  $v_i$  y un peso  $w_i$ . La mochila posee una capacidad máxima de peso  $W$ . Se define una variable binaria  $x_i$  para cada ítem:

$$x_i = \begin{cases} 1, & \text{si el ítem } i \text{ es seleccionado,} \\ 0, & \text{si el ítem } i \text{ no es seleccionado.} \end{cases}$$

La formulación clásica del problema es:

$$\begin{aligned} \text{Maximizar} \quad & Z = \sum_{i=1}^n v_i x_i \\ \text{sujeto a} \quad & \sum_{i=1}^n w_i x_i \leq W, \\ & x_i \in \{0, 1\}, \quad i = 1, 2, \dots, n. \end{aligned}$$

En esta actividad, la mochila clásica se utilizará como punto de partida conceptual, pero la implementación deberá resolver una versión extendida del problema y paralelizar sus componentes principales mediante CUDA.

### 3. Versión extendida de la mochila

Para esta actividad no se trabajará únicamente con la mochila 0/1 clásica. Cada grupo deberá implementar una versión extendida, donde cada ítem posee:

- **ID:** identificador único del ítem.
- **Valor:** beneficio asociado al ítem.
- **Peso:** peso ocupado por el ítem.
- **Volumen:** espacio ocupado por el ítem.
- **Categoría:** grupo al que pertenece el ítem.

Además, la mochila tendrá restricciones adicionales:

1. **Restricción de peso:** la suma de pesos no debe superar  $W$ .
2. **Restricción de volumen:** la suma de volúmenes no debe superar  $V$ .
3. **Restricciones por categoría:** algunas categorías pueden tener un mínimo o máximo de ítems seleccionados.
4. **Incompatibilidades:** ciertos pares de ítems no pueden seleccionarse simultáneamente.
5. **Dependencias:** si se selecciona un ítem, puede ser obligatorio seleccionar otro ítem asociado.

Por lo tanto, la solución ya no será válida solamente por cumplir el peso máximo. También deberá respetar volumen, categorías, incompatibilidades y dependencias.

## 4. Representación de una solución

Cada individuo del algoritmo genético será representado como un cromosoma binario de largo  $n$ :

$$X = (x_1, x_2, x_3, \dots, x_n)$$

Donde cada posición indica si el ítem correspondiente está incluido o no en la mochila.

Ejemplo:

|                                  |
|----------------------------------|
| <code>X = 1 0 1 1 0 0 1 0</code> |
|----------------------------------|

En este caso, se seleccionan los ítems 0, 2, 3 y 6.

Para la implementación en CUDA, el grupo deberá decidir y justificar la representación de la población en memoria. Se recomienda considerar una representación lineal de la población y evaluar si conviene usar una estructura por individuo o una estructura orientada a accesos coalescentes.

Ejemplo conceptual de indexación en GPU:

|                                                                                                                           |
|---------------------------------------------------------------------------------------------------------------------------|
| <pre>global_individual = blockIdx.x * blockDim.x + threadIdx.x; gene_index = global_individual * n_items + item_id;</pre> |
|---------------------------------------------------------------------------------------------------------------------------|

## 5. Función de aptitud

La función de aptitud debe considerar el valor total de la solución y penalizar las restricciones incumplidas. Una forma recomendada es:

$$\text{fitness}(X) = \text{valor\_total}(X) - \text{penalización\_total}(X)$$

Una penalización posible es:

$$\begin{aligned} \text{fitness}(X) = & \sum_{i=1}^n v_i x_i - \alpha \cdot \text{exceso\_peso}(X) - \beta \cdot \text{exceso\_volumen}(X) \\ & - \gamma \cdot \text{violaciones\_categoría}(X) - \delta \cdot \text{incompatibilidades}(X) - \varepsilon \cdot \text{dependencias\_incumplidas}(X) \end{aligned}$$

Los valores de  $\alpha$ ,  $\beta$ ,  $\gamma$ ,  $\delta$  y  $\varepsilon$  deberán ser definidos y justificados por cada grupo. Una penalización demasiado baja puede permitir soluciones inválidas; una penalización excesivamente alta puede impedir explorar soluciones cercanas a la factibilidad.

En CUDA, la evaluación de aptitud debe ser tratada como uno de los núcleos principales de paralelización. El grupo deberá explicar si su estrategia usa un hilo por individuo, un bloque por individuo, reducciones internas o alguna combinación de estas alternativas.

## 5.1 Restricciones duras y restricciones blandas

Para esta actividad se deberá distinguir entre restricciones duras y restricciones blandas.

- **Restricciones duras:** son aquellas que la solución final reportada debe cumplir obligatoriamente. En esta actividad, se consideran restricciones duras el peso máximo  $W$ , el volumen máximo  $V$ , las incompatibilidades entre ítems y las dependencias obligatorias entre ítems.
- **Restricciones blandas:** son condiciones que pueden ser incumplidas temporalmente durante la evolución del algoritmo genético, pero que deben recibir una penalización en la función de aptitud. Las restricciones por categoría podrán ser tratadas como blandas cuando el grupo justifique que desea favorecer ciertas composiciones sin descartar inmediatamente soluciones cercanas.

Durante la ejecución del algoritmo genético se permitirá que existan individuos que violen una o más restricciones. Estos individuos no deberán ser eliminados automáticamente, sino evaluados mediante penalizaciones. Esto permite que el algoritmo explore soluciones cercanas a regiones factibles y que, mediante cruzamiento o mutación, puedan transformarse en soluciones válidas de buena calidad.

Sin embargo, la solución final informada por cada grupo deberá ser factible respecto de las restricciones duras. Por lo tanto, si el individuo con mayor *fitness* viola alguna restricción dura, el grupo deberá reportar como resultado final la mejor solución factible encontrada durante la ejecución.

En el informe se deberá indicar explícitamente:

- qué restricciones fueron tratadas como duras;
- qué restricciones fueron tratadas como blandas;
- cómo se calculó la penalización asociada a cada violación;
- cómo se verificó que la solución final reportada es factible.

## 6. Variantes obligatorias

Cada grupo deberá implementar y comparar las siguientes variantes.

## 6.1 Variante 1: Algoritmo genético secuencial en CPU

Debe incluir, como mínimo:

- Generación de población inicial.
- Evaluación de aptitud.
- Selección por torneo.
- Cruzamiento.
- Mutación.
- Elitismo.
- Criterio de término por número de generaciones o convergencia.

Esta variante será la línea base para comparar el rendimiento de las versiones CUDA.

## 6.2 Variante 2: Algoritmo genético CUDA básico

Se debe implementar una versión funcional en CUDA. Esta versión debe paralelizar, analizar y justificar al menos las siguientes partes:

- Evaluación de la función de aptitud para la población.
- Selección por torneo dentro de la población correspondiente.
- Aplicación de operadores de cruzamiento y mutación.
- Actualización de la nueva generación.
- Búsqueda o registro del mejor individuo factible.

La población debe mantenerse preferentemente en memoria de GPU durante la evolución del algoritmo. No se recomienda transferir la población completa entre CPU y GPU en cada generación, ya que esto puede ocultar o destruir el beneficio de la paralelización.

El informe deberá explicar qué kernels fueron implementados, qué datos permanecen en memoria global, qué datos se copian desde y hacia el host, cómo se controla la aleatoriedad y cómo se evitan condiciones de carrera.

## 6.3 Variante 3: Algoritmo genético CUDA optimizado

Se debe implementar una versión CUDA optimizada y compararla con la versión CUDA básica. La optimización no debe limitarse obligatoriamente al uso de *shared memory*; debe justificarse de acuerdo con el comportamiento real del algoritmo y de la arquitectura GPU utilizada.

Cada grupo debe integrar y justificar técnicamente al menos 5 de las siguientes estrategias, demostrando con mediciones cuáles generan mejoras reales de rendimiento:

- Uso justificado de *shared memory* cuando exista reutilización de datos dentro de un bloque.
- Uso de memoria constante para parámetros o datos de solo lectura.
- Accesos coalescentes a memoria global.
- Reducciones paralelas para calcular fitness, pesos, volúmenes o mejores individuos.
- Disminución de transferencias entre host y device.
- Ajuste del tamaño de bloque y número de bloques.
- Uso de *streams* CUDA, si corresponde.
- Control de divergencia de *warps*.
- Manejo adecuado de números aleatorios por hilo, individuo o generación.

No se evaluará positivamente el uso artificial de *shared memory* si no existe una razón técnica clara. El grupo deberá demostrar qué optimizaciones mejoran el rendimiento y cuáles no aportan mejoras significativas.

## 7. Diseño experimental mínimo

Cada grupo deberá ejecutar experimentos suficientes para comparar rendimiento y calidad de solución. Como mínimo, se solicita:

- **Tres tamaños de instancia:** pequeña, mediana y grande.
- **Tamaños de población:** al menos tres configuraciones, por ejemplo 1.024, 4.096 y 16.384 individuos.
- **Variantes:** CPU secuencial, CUDA básica y CUDA optimizada.
- **Repeticiones:** al menos 10 ejecuciones por cada configuración experimental relevante.
- **Semillas:** las ejecuciones deben usar semillas registradas para permitir reproducibilidad.
- **Hardware:** se debe reportar CPU, GPU, memoria disponible, versión de CUDA y versión del driver.

Tamaños sugeridos:

| Instancia | Cantidad de ítems sugerida | Objetivo                               |
|-----------|----------------------------|----------------------------------------|
| Pequeña   | 100 ítems                  | Verificación funcional y depuración    |
| Mediana   | 1.000 ítems                | Comparación básica de rendimiento      |
| Grande    | 10.000 ítems               | Evaluación real del paralelismo en GPU |

Para evitar restricciones triviales o imposibles, la capacidad máxima de peso  $W$  y la capacidad máxima de volumen  $V$  deberán calcularse como un porcentaje del peso total y del volumen total de todos los ítems disponibles.

Se recomienda utilizar:

$$W = 0,40 \times \text{suma\_total\_pesos}$$

$$V = 0,40 \times \text{suma\_total\_volúmenes}$$

Para instancias fáciles podrá usarse un porcentaje entre 50 % y 60 %. Para instancias medias, entre 35 % y 50 %. Para instancias difíciles, entre 20 % y 35 %.

No se aceptarán configuraciones donde la capacidad permita seleccionar casi todos los ítems, ni configuraciones donde no exista una solución factible.

## 8. Métricas obligatorias

El informe debe incluir, como mínimo, las siguientes métricas:

- Tiempo promedio de ejecución total.
- Desviación estándar del tiempo total.
- Tiempo de kernels CUDA.
- Tiempo de transferencia host-device y device-host, cuando corresponda.
- Mejor valor factible encontrado.
- Mejor *fitness* obtenido.
- Porcentaje de soluciones factibles.
- *Speed-up* respecto de la versión CPU secuencial.
- Comparación entre CUDA básica y CUDA optimizada.
- Efecto del tamaño de bloque.
- Efecto del tamaño de población.
- Calidad de solución versus tiempo de ejecución.

Nota: el *speed-up* debe calcularse comparando el tiempo de la versión CPU secuencial contra el tiempo de la versión CUDA correspondiente. El grupo debe indicar si el tiempo CUDA incluye solo kernels o también transferencias entre host y device.

## 9. Formato sugerido de entrada

Cada grupo puede diseñar su propio formato de entrada, pero debe documentarlo claramente. Se recomienda trabajar con archivos separados:

```
items.csv
id,valor,peso,volumen,categoria

category_rules.csv
categoria,minimo,maximo

incompatibilities.csv
id_item_a,id_item_b

dependencies.csv
id_item,id_requerido
```

El programa deberá permitir seleccionar instancia, variante, tamaño de población, número de generaciones, tamaño de bloque y semilla desde la línea de comandos.

## 10. Entregables

Cada grupo deberá entregar:

1. Código fuente en CUDA C++ y C++ para la versión secuencial.
2. README con instrucciones claras de compilación y ejecución.
3. Instancias utilizadas o generador de instancias.
4. Archivo CSV con los resultados experimentales.
5. Informe en PDF.
6. Presentación en formato PPTX utilizada en la defensa. Puede adjuntarse además una versión PDF de respaldo.

La entrega del código es obligatoria. Durante la presentación, cada grupo deberá mostrar la ejecución del programa, presentar los resultados generados y explicar de forma simple pero técnicamente correcta cómo se implementaron las partes principales del algoritmo. Los detalles importantes deben estar incorporados en la PPTX: por ejemplo, cómo se paralelizó la evaluación de aptitud, cómo se implementó la selección por torneo, cómo se realizaron cruzamiento y mutación, qué partes se ejecutan en CPU/GPU y qué datos se transfieren entre host y device. No se espera que el grupo abra el código completo durante la exposición; puede mostrar fragmentos breves de funciones o kernels si estos están insertos en la presentación y ayudan a justificar una decisión técnica. La ejecución podrá realizarse localmente o mediante acceso remoto a una máquina con GPU. No se considerará suficiente mostrar únicamente capturas estáticas sin ejecutar ni explicar los resultados.

Se recomienda una estructura de carpetas como la siguiente:



```
actividad_mochila_cuda/  
|-- src/  
| |-- main.cu  
| |-- genetic_algorithm_cpu.cpp  
| |-- genetic_algorithm_cpu.hpp  
| |-- genetic_algorithm_cuda.cu  
| |-- genetic_algorithm_cuda.cuh  
| |-- fitness_cuda.cu  
| |-- fitness_cuda.cuh  
| |-- instance_loader.cpp  
| |-- instance_loader.hpp  
|  
|-- data/  
| |-- small/  
| |-- medium/  
| |-- large/  
|  
|-- results/  
| |-- resultados.csv  
|  
|-- report/  
| |-- informe.pdf  
|  
|-- presentation/  
| |-- presentacion.pptx  
|  
|-- README.md
```

Compilación sugerida:

```
nvcc -O3 src/*.cu src/*.cpp -o mochila_ga_cuda
```

Si el grupo utiliza una estructura de proyecto más compleja, puede entregar un **Makefile** o un archivo de configuración **CMakeLists.txt**, siempre que el **README** explique claramente cómo compilar y ejecutar.

## 11. Condiciones de evaluación durante la presentación

La presentación debe explicar los aspectos técnicos principales de manera simple, ordenada y visual. Los detalles relevantes de la implementación deben estar incorporados en la PPTX mediante esquemas, pseudocódigo, tablas, diagramas de flujo, fragmentos breves de kernels o funciones, y resultados experimentales.

Durante la presentación y defensa, cada grupo deberá:

- Introducción al problema de la mochila de forma breve.
- Mostrar al menos una ejecución reproducible, indicando instancia, variante, tamaño de población, generaciones, tamaño de bloque y semilla.
- Mostrar resultados generados por el programa, por ejemplo salida en consola o archivo CSV.
- Justificar cómo se implementan las restricciones, la función de aptitud (Fitness), la selección, el cruzamiento, la mutación y el elitismo.
- Explicar qué partes se ejecutan en CPU, qué partes se ejecutan en GPU y qué datos se transfieren entre host y device.
- Responder preguntas técnicas sobre memoria, indexación, aleatoriedad, sincronización, transferencias, medición de tiempos y correctitud de resultados.

No se espera que los estudiantes abran el código completo durante la presentación. Si se desea mostrar código, este debe aparecer como un fragmento breve dentro de la PPTX y debe estar acompañado de una explicación clara de su función. La presentación no debe transformarse en una lectura de código, sino en una explicación de las decisiones de diseño y paralelización.

Si el programa no ejecuta o no puede ser demostrado durante la presentación, el grupo no podrá obtener el puntaje máximo en los criterios asociados a ejecución, correctitud funcional, implementación CUDA y defensa técnica, aunque el informe esté completo.

## 12. Pauta de evaluación del informe técnico

La siguiente pauta corresponde al **30 % de la actividad**. Cada criterio se evalúa con puntaje entero de 0 a 3 y luego se pondera según el porcentaje indicado dentro del informe.

| Criterio                                       | Ponderación | 0 puntos                                                       | 1 punto                                                                                                                          | 2 puntos                                                                                                              | 3 puntos                                                                                                                                                                                                      |
|------------------------------------------------|-------------|----------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Introducción, contexto y formulación extendida | 20 %        | No presenta el problema o la formulación es inexistente.       | Presenta una descripción general o solo formula la mochila clásica.                                                              | Presenta el problema extendido, aunque con algunas restricciones incompletas o poco formalizadas.                     | Contextualiza claramente la mochila, su complejidad, aplicaciones, variables, función objetivo, restricciones extendidas, restricciones duras, restricciones blandas y factibilidad final.                    |
| Metodología y diseño experimental              | 20 %        | No presenta metodología ni diseño experimental claro.          | Presenta experimentos insuficientes o sin control de semillas, instancias, población, generaciones, tamaño de bloque o hardware. | Presenta un diseño aceptable, pero con alguna omisión en repeticiones, variantes, configuración CUDA o procedimiento. | Define claramente instancias, variantes, población, generaciones, tamaño de bloque, semillas, repeticiones, hardware, versión CUDA, driver y procedimiento experimental.                                      |
| Descripción del algoritmo genético y variantes | 15 %        | No describe el algoritmo implementado.                         | Describe etapas aisladas de forma superficial.                                                                                   | Describe las etapas principales, pero con bajo detalle de parámetros o diferencias entre variantes.                   | Explica representación, población, función de aptitud, selección, cruzamiento, mutación, elitismo, criterio de término y diferencias entre CPU, CUDA básica y CUDA optimizada.                                |
| Resultados, tablas y gráficos                  | 20 %        | No presenta resultados cuantitativos.                          | Presenta resultados incompletos, poco claros o sin gráficos relevantes.                                                          | Presenta tablas y gráficos suficientes, aunque con interpretación limitada o ausencia de algunas métricas.            | Presenta resultados completos, ordenados y reproducibles, incluyendo tiempos, desviación estándar, mejor valor factible, mejor fitness, factibilidad, speed-up, comparación CPU/GPU y CUDA básica/optimizada. |
| Análisis técnico y conclusiones                | 20 %        | No analiza los resultados o las conclusiones son irrelevantes. | Entrega una interpretación superficial, sin relacionar resultados con CUDA o calidad de solución.                                | Analiza parcialmente rendimiento, memoria, transferencias, kernels u optimizaciones, pero con vacíos técnicos.        | Discute críticamente rendimiento, overhead, transferencias, uso de memoria, tamaño de bloque, escalabilidad, calidad de solución, límites observados y aprendizajes técnicos.                                 |

| Criterio                        | Ponderación  | 0 puntos                                                                                      | 1 punto                                                                            | 2 puntos                                                                | 3 puntos                                                                                                                                                                |
|---------------------------------|--------------|-----------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|-------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Redacción, orden y trazabilidad | 5 %          | Informe desordenado, incompleto o difícil de seguir.                                          | Presenta problemas importantes de redacción, formato o trazabilidad de resultados. | Informe comprensible, con formato aceptable y pequeñas inconsistencias. | Informe claro, ordenado, bien redactado, con tablas y gráficos correctamente rotulados, parámetros trazables y coherencia entre metodología, resultados y conclusiones. |
| <b>Total</b>                    | <b>100 %</b> | Este total corresponde internamente al informe técnico, que equivale al 30 % de la actividad. |                                                                                    |                                                                         |                                                                                                                                                                         |

## 13. Pauta integrada de presentación, defensa y ejecución del código

La siguiente pauta corresponde al **70 % de la actividad**. En esta instancia se evalúa la presentación, la defensa técnica, la ejecución del programa y la explicación de la implementación. No se exige abrir el código completo durante la exposición; los elementos técnicos relevantes deben estar explicados en la PPTX y pueden apoyarse con fragmentos breves de código cuando sea necesario.

| Criterio                                                            | Ponderación | 0 puntos                                                                    | 1 punto                                                                                                             | 2 puntos                                                                                                                                              | 3 puntos                                                                                                                                                                                                                    |
|---------------------------------------------------------------------|-------------|-----------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Claridad del problema y propuesta                                   | 8 %         | No explica el problema o lo presenta incorrectamente.                       | Explica solo la mochila clásica o presenta una motivación débil.                                                    | Explica el problema extendido, aunque con vacíos sobre restricciones, factibilidad o dificultad.                                                      | Explica claramente objetivo, variables, restricciones duras, restricciones blandas, factibilidad final, dificultad del problema y sentido de usar paralelismo GPU.                                                          |
| Explicación técnica en la PPTX del algoritmo y de la implementación | 18 %        | No explica el algoritmo ni la función de las partes principales del código. | Explica etapas aisladas, sin conectar adecuadamente algoritmo, implementación y variantes.                          | Explica las etapas principales mediante la PPTX, pero con detalles incompletos sobre selección, cruzamiento, mutación, elitismo o flujo de ejecución. | Explica de forma clara y simple, usando la PPTX, cómo se implementan población, fitness, restricciones, selección por torneo, cruzamiento, mutación, elitismo, criterio de término, variantes y flujo general del programa. |
| Ejecución reproducible y resultados generados                       | 18 %        | No ejecuta el programa o la demostración falla sin explicación técnica.     | Ejecuta parcialmente, con comandos incompletos, parámetros poco claros o sin reproducibilidad.                      | Ejecuta una o más variantes y muestra resultados, pero con pequeñas omisiones en parámetros, semillas, instancia o validación.                        | Muestra una ejecución reproducible indicando instancia, variante, tamaño de población, generaciones, tamaño de bloque y semilla; además muestra salida en consola o archivo CSV generado por el programa.                   |
| Correctitud funcional y modelado de restricciones                   | 14 %        | El programa no resuelve el problema o produce resultados inválidos.         | Resuelve una versión incompleta, con errores relevantes en restricciones, fitness, operadores o factibilidad final. | Resuelve la mayor parte del problema extendido, con errores menores o casos borde no tratados.                                                        | Demuestra funcionalmente la mochila extendida, penalizaciones, restricciones duras/blandas, mejor solución factible, operadores genéticos y consistencia entre salida, CSV, informe y presentación.                         |

| Criterio                                                   | Ponderación  | 0 puntos                                                                                                                                          | 1 punto                                                                                                            | 2 puntos                                                                                                        | 3 puntos                                                                                                                                                                                                                                                                           |
|------------------------------------------------------------|--------------|---------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Implementación CUDA, distribución CPU/GPU y transferencias | 18 %         | No implementa CUDA de forma funcional o no identifica qué se ejecuta en GPU.                                                                      | Presenta kernels superficiales, con errores de indexación, memoria, sincronización, aleatoriedad o transferencias. | Implementa kernels funcionales y explica parte de la distribución CPU/GPU, aunque con justificación incompleta. | Explica qué partes se ejecutan en CPU, qué partes se ejecutan en GPU, qué datos se transfieren entre host y device, y justifica memoria global, shared memory cuando corresponda, memoria constante, indexación, aleatoriedad, sincronización y control de condiciones de carrera. |
| Optimización, medición y comparación de variantes          | 12 %         | No presenta optimización ni mediciones confiables.                                                                                                | Presenta optimizaciones declaradas, pero sin evidencia de mejora o sin medición separada.                          | Compara CPU, CUDA básica y CUDA optimizada con resultados aceptables, aunque con análisis limitado.             | Demuestra y explica optimizaciones, medición de kernels, transferencias, tiempos totales, speed-up, efecto del tamaño de bloque, tamaño de población y relación entre rendimiento y calidad de solución.                                                                           |
| Defensa técnica individual y participación del grupo       | 12 %         | No responde preguntas técnicas básicas o la presentación recae en una sola persona.                                                               | Responde parcialmente, con alta dependencia de terceros o participación muy desigual.                              | Responde la mayoría de las preguntas con fundamentos aceptables y participación relativamente equilibrada.      | Todos los integrantes participan y responden con seguridad sobre memoria, indexación, aleatoriedad, sincronización, transferencias, medición de tiempos, correctitud de resultados, fórmulas, CUDA y decisiones técnicas.                                                          |
| <b>Total</b>                                               | <b>100 %</b> | Este total corresponde internamente a la presentación, defensa, ejecución y explicación técnica del código, que equivale al 70 % de la actividad. |                                                                                                                    |                                                                                                                 |                                                                                                                                                                                                                                                                                    |